

DFS:

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```
visited = set()
```

```
def dfs(node):
```

```
    if node not in visited:  
        print(node, end=' ')  
        visited.add(node)  
        for neighbor in graph[node]:  
            dfs(neighbor)
```

```
print("DFS starting from A:")
```

```
dfs('A')
```

BFS:

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []
```

```
}
```

```
visited = set()
```

```
def bfs(start):
```

```
    queue = deque([start])
```

```
    while queue:
```

```
        node = queue.popleft()
```

```
        if node not in visited:
```

```
            print(node, end=' ')
```

```
            visited.add(node)
```

```
            for neighbor in graph[node]:
```

```
                queue.append(neighbor)
```

```
print("BFS starting from A:")
```

```
bfs('A')
```

8-Queens :

```
def print_board(q):
```

```
    for row in q:
```

```
        line = ['.'] * 8
```

```
        line[row] = 'Q'
```

```
        print(' '.join(line))
```

```
    print()
```

```
def solve(q=[], row=0):
```

```
    if row == 8:
```

```
        print_board(q)
```

```
        return True
```

```
    for col in range(8):
```

```
        if all(col != c and abs(row - r) != abs(col - c) for r, c in enumerate(q)):
```

```

    if solve(q + [col], row + 1):
        return True

```

```

solve()

```

Vacuum Cleaner:

```

rooms = {'A': 'Dirty', 'B': 'Dirty'}
loc = 'A'

```

```

for _ in range(4):
    if rooms[loc] == 'Dirty':
        print("Location", loc + ":", "Suck")
        rooms[loc] = 'Clean'
    else:
        print("Location", loc + ":", "Clean")
    loc = 'B' if loc == 'A' else 'A'

```

Alpha Beta Pruning:

```

def alphabeta(node, a, b, maxP):
    if type(node) == int:
        return node
    if maxP:
        return max(alphabeta(c, a, b, False) for c in node)
    else:
        return min(alphabeta(c, a, b, True) for c in node)

```

```

tree = [[3,5,6],[2,[0,-1],4],[1,2]]

```

```

print(alphabeta(tree, float('-inf'), float('inf'), True))

```

Travelling Sales Person / a-star:

```

from itertools import permutations

```

```
def tsp(dist):  
    n = len(dist)  
    cities = range(n)  
    best = min(  
        (sum(dist[path[i]][path[i+1]] for i in range(n-1)) + dist[path[-1]][path[0]], path)  
        for path in permutations(cities)  
    )  
    return best
```

```
# Example distance matrix (symmetric)
```

```
dist = [  
    [0, 1, 3, 4],  
    [1, 0, 2, 5],  
    [3, 2, 0, 6],  
    [4, 5, 6, 0]  
]
```

```
cost, route = tsp(dist)  
print("Cost:", cost)  
print("Route:", route)
```